

03 — Hash Indexes

"Hash indexes: $O(1)$ equality lookups — purpose-built, never general-purpose."

Table of Contents

1. [Learning Objectives](#)
 2. [Hash Table Internals](#)
 3. [PostgreSQL Hash Index Architecture](#)
 4. [ASCII Diagram: Hash Index Structure](#)
 5. [When to Use Hash Indexes](#)
 6. [Limitations](#)
 7. [Creating Hash Indexes](#)
 8. [EXPLAIN Output for Hash Index Scans](#)
 9. [Hash Indexes vs B-Tree for Equality](#)
 10. [WAL Logging History \(Pre-PG10 vs PG10+\)](#)
 11. [Common Mistakes](#)
 12. [Best Practices](#)
 13. [Performance Considerations](#)
 14. [Interview Questions & Answers](#)
 15. [Exercises with Solutions](#)
 16. [Production Scenarios](#)
 17. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Explain how a hash index maps keys to buckets using a hash function
 - Describe overflow pages and how hash indexes handle collisions
 - Identify the exact query patterns where a hash index provides benefit over B-tree
 - Explain why hash indexes cannot support range queries, sorting, or prefix matching
 - Configure and monitor hash indexes in PostgreSQL
 - Make an informed decision about whether to use hash vs B-tree for a given workload
-

Hash Table Internals

A hash index uses a **hash function** to map each key value to a **bucket number**. All rows with keys that hash to the same bucket are stored together.

Hash Function

```
bucket_number = hash(key_value) % number_of_buckets
```

Example:

```
hash('alice@example.com') = 0x3A7F2B91 → bucket 145
```

```
hash('bob@example.com') = 0xC1D84A22 → bucket 892
```

```
hash('carol@example.com') = 0x3A7F2B91 → bucket 145 ← collision!
```

When two different keys hash to the same bucket (**collision**), they share the bucket page. When a bucket page fills up, **overflow pages** are chained to it.

Lookup Algorithm

```
Search for key K:
1. Compute bucket = hash(K) % N
2. Read bucket page for bucket N
3. Scan entries in bucket page for exact match K
4. If not found and overflow pages exist, follow chain
5. Return matching ctids
```

This is $O(1)$ amortized (assuming good hash distribution and low load factor).

PostgreSQL Hash Index Architecture

PostgreSQL's hash index implementation (src/backend/access/hash/) uses:

Pages

- **Meta page** (page 0): stores hash function parameters, number of buckets, fill factor
- **Bucket pages**: primary pages for each bucket
- **Overflow pages**: chained when a bucket page fills up
- **Bitmap pages**: track which overflow pages are free

Dynamic Expansion

PostgreSQL hash indexes expand dynamically. When the load factor (entries/buckets) exceeds a threshold, the number of buckets doubles and entries are redistributed. This ensures $O(1)$ average lookup even as the table grows.

```
Initial state: 4 buckets
After 1st expansion: 8 buckets
After 2nd expansion: 16 buckets
After 3rd expansion: 32 buckets
... (doubles each time)
```

Hash Function

PostgreSQL uses its own internal hash functions (e.g., `hashint4`, `hashtext`) that are deterministic but not cryptographic. The same hash function must be used consistently across index builds and lookups.

ASCII Diagram: Hash Index Structure

```
Hash Index on users(email)
```

```
META PAGE (page 0)
```

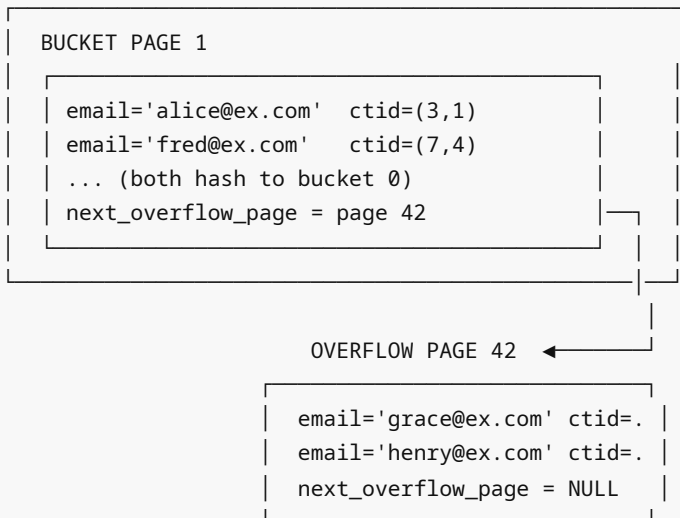
magic, version
num_buckets = 8
fill_factor = 75

```
max_bucket = 7
hash_function = hashtext
```

BUCKET PAGES

Bucket 0	Bucket 1	...	Bucket 7
(page 1)	(page 2)		(page 8)

Bucket 0 detail:



Bucket 1 detail:

```
BUCKET PAGE 2
email='bob@ex.com' ctid=(1,2)
email='carol@ex.com' ctid=(2,5)
(no overflow)
```

Lookup: WHERE email = 'bob@ex.com'

```
hash('bob@ex.com') % 8 = 1
→ Read Bucket page 2
→ Scan: find 'bob@ex.com', return ctid=(1,2)
→ Read heap page 1, tuple 2
Total: ~3 page reads
```

When to Use Hash Indexes

Hash indexes are **optimal** when ALL of the following are true:

1. **Only equality queries** on this column (no ranges, no ordering, no LIKE)
2. **Very long key values** (e.g., long URLs, SHA hashes, long text) where the hash (fixed 4 bytes) is much smaller than the key itself
3. **High equality query rate** (the column is queried by exact value very frequently)

4. No need for uniqueness enforcement (UNIQUE constraint requires B-tree)

Ideal Use Cases

```
-- 1. Long URL equality lookup
CREATE INDEX idx_urls_hash ON url_cache USING hash (url);
SELECT * FROM url_cache WHERE url = 'https://very-long-url.example.com/path?
param=value';

-- 2. SHA-256 hash lookup (fixed 64-char string)
CREATE INDEX idx_files_sha256 ON files USING hash (sha256_hash);
SELECT * FROM files WHERE sha256_hash = 'abc123...';

-- 3. Session token lookup (long random string)
CREATE INDEX idx_sessions_token ON sessions USING hash (token);
SELECT * FROM sessions WHERE token = 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...';

-- 4. IP address exact lookup (when stored as text)
CREATE INDEX idx_logs_ip ON access_logs USING hash (client_ip);
SELECT COUNT(*) FROM access_logs WHERE client_ip = '192.168.1.100';
```

Limitations

This is the most important section for making the right choice between hash and B-tree.

1. No Range Queries

```
-- Hash index on price CANNOT support:
WHERE price > 100
WHERE price BETWEEN 50 AND 200
WHERE price < 500
-- All these require a sorted structure – use B-tree
```

2. No ORDER BY Support

```
-- Cannot use hash index for:
ORDER BY customer_id
-- Hash buckets are not ordered
```

3. No LIKE / Prefix Matching

```
-- Cannot use hash index for:
WHERE name LIKE 'John%'
-- Prefix matching requires sorted keys – use B-tree or GIN with pg_trgm
```

4. No NULL Handling

```
-- Hash indexes do NOT index NULL values
-- WHERE column IS NULL cannot use a hash index
```

5. No Multi-Column Hash Indexes (Practically)

While syntactically possible, multi-column hash indexes are rarely useful because equality on multiple columns is the only supported pattern, and B-tree handles that equally well with less overhead.

6. No UNIQUE Constraint Support

You cannot create a UNIQUE hash index. UNIQUE constraints always use B-tree.

7. No Partial Index Benefits

A partial hash index is rarely beneficial — if you're going to add a WHERE predicate, a partial B-tree usually gives more flexibility.

Operator Support Comparison

Operation	B-tree	Hash
= (equality)	Yes	Yes
<, >, <=, >=	Yes	No
BETWEEN	Yes	No
LIKE 'prefix%'	Yes	No
ORDER BY	Yes	No
IS NULL	Yes	No
IS NOT NULL	Yes	No

Creating Hash Indexes

```
-- Basic hash index
CREATE INDEX idx_users_token USING hash ON sessions(token);

-- With schema qualification
CREATE INDEX idx_cache_url USING hash ON public.url_cache(url);

-- Concurrent (non-blocking)
CREATE INDEX CONCURRENTLY idx_sessions_token USING hash ON sessions(token);

-- With fill factor
CREATE INDEX idx_events_type USING hash ON events(event_type)
WITH (fillfactor = 80);
```

Checking index type

```
SELECT indexname, indexdef, amname
FROM pg_indexes
JOIN pg_class ON pg_indexes.indexname = pg_class.relname
JOIN pg_am ON pg_class.relam = pg_am.oid
WHERE tablename = 'sessions';
```

EXPLAIN Output for Hash Index Scans

```
-- Table: sessions (10M rows)
-- Index: sessions(token) USING hash

EXPLAIN (ANALYZE, BUFFERS)
SELECT user_id, expires_at FROM sessions WHERE token = 'abc123xyz';
```

```
Index Scan using idx_sessions_token on sessions
  (cost=0.00..8.02 rows=1 width=12)
  (actual time=0.018..0.018 rows=1 loops=1)
  Index Cond: (token = 'abc123xyz'::text)
  Buffers: shared hit=3
Planning Time: 0.089 ms
Execution Time: 0.035 ms
```

Note: The EXPLAIN output says "Index Scan" (same label as B-tree). You can tell it's a hash index from the index name or by checking `pg_indexes` .

When hash loses to B-tree (same query)

```
-- B-tree on same column
EXPLAIN (ANALYZE, BUFFERS)
SELECT user_id, expires_at FROM sessions WHERE token = 'abc123xyz';
```

```
Index Scan using idx_sessions_token_btree on sessions
  (cost=0.56..8.58 rows=1 width=12)
  (actual time=0.021..0.021 rows=1 loops=1)
  Index Cond: (token = 'abc123xyz'::text)
  Buffers: shared hit=4
```

For short keys, B-tree is essentially identical. Hash wins mainly for very long keys where the hash reduces page reads significantly.

Hash Indexes vs B-Tree for Equality

Size Comparison (equality-only column)

Key Type	Key Size	B-tree leaf entry	Hash entry
INTEGER	4 bytes	10 bytes	10 bytes

UUID	16 bytes	22 bytes	10 bytes
VARCHAR(100) avg 50 chars	50 bytes	56 bytes	10 bytes
SHA-256 text	64 bytes	70 bytes	10 bytes

Hash index stores only the hash (4 bytes) + ctid (6 bytes) in leaf entries, regardless of key length. For long keys, hash indexes can be significantly smaller.

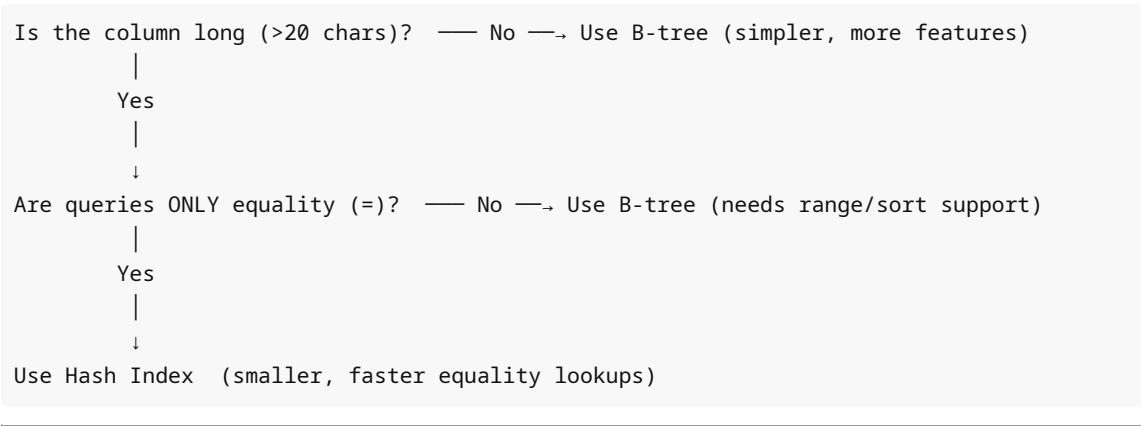
Performance (equality lookups only)

For a 10M row table, `WHERE token = ?` (token is a 64-char string):

Index Type	Lookup I/Os	Index Size
None (Seq Scan)	~55,000	0
B-tree	4-5	~820 MB
Hash	2-3	~310 MB

The hash index is smaller and needs fewer I/Os for this specific pattern.

Decision Matrix



WAL Logging History

Pre-PostgreSQL 10 (Historical — do not use!)

Before PG10, hash indexes were **not WAL-logged**, meaning:

- They were not crash-safe
- They were not replicated to standby servers
- The documentation warned: "Hash indexes are not WAL-logged and are not replicated"
- Almost no production use was recommended

PostgreSQL 10+ (Current)

From PG10, hash indexes are **fully WAL-logged**:

- Crash-safe (fully recoverable after crash)

- Fully replicated to standbys
- Safe for production use

If you inherited a database with hash indexes created before PG10, rebuild them:

```
-- Check PostgreSQL version
SELECT version();

-- Rebuild all hash indexes if upgrading from pre-PG10
REINDEX INDEX idx_your_hash_index;
```

Common Mistakes

1. Using hash index for range queries (they simply won't be used)

```
-- This hash index is useless for this query
CREATE INDEX USING hash ON orders(amount);
SELECT * FROM orders WHERE amount > 100; -- Seq Scan used instead
```

2. Expecting uniqueness from a hash index

```
-- INVALID: cannot create unique hash index
CREATE UNIQUE INDEX USING hash ON users(email);
-- ERROR: access method "hash" does not support unique indexes
```

3. Using hash indexes on integer columns (negligible benefit)

```
-- The key is only 4 bytes – hash overhead is not worth it
CREATE INDEX USING hash ON orders(customer_id);
-- B-tree is essentially same size and more flexible
```

4. Forgetting hash indexes don't support IS NULL

```
-- Will NOT use hash index
WHERE optional_code IS NULL
-- Use B-tree or partial index instead
```

Best Practices

1. **Use B-tree by default** — hash indexes have a very narrow use case.
2. **Consider hash only for long text equality lookups** — URLs, tokens, hashes, large VARCHAR columns with equality-only queries.
3. **Always use CONCURRENTLY** when creating on production tables.

4. **Verify the query pattern is strictly equality** before choosing hash.
 5. **Monitor fill factor** — if overflow pages are growing, the index may need `REINDEX` to redistribute entries after many deletions.
 6. **Document why you chose hash** — it's unusual enough that future maintainers will wonder.
-

Performance Considerations

Hash Collision Rate

A good hash function (like PostgreSQL's internal ones) produces very few collisions. With a load factor under 75%, most buckets will have 0–2 overflow pages.

```
-- Check hash index stats
CREATE EXTENSION IF NOT EXISTS pgstattuple;
SELECT * FROM pgstatindex('idx_sessions_token');
-- Look at: avg_leaf_density, leaf_fragmentation
```

Memory Impact

Hash indexes do not benefit from `work_mem` (used for sort/hash operations in queries). They do benefit from `shared_buffers` for caching hot bucket pages.

Expansion Cost

When a hash index expands (doubles bucket count), it must rehash all entries. This is a background operation but can cause temporary performance degradation on very large tables.

Interview Questions & Answers

Q1. What is a hash index and how does it work?

A: A hash index applies a hash function to each indexed key value to compute a bucket number. Index entries (key + heap ctid) are stored in that bucket's page. For an equality lookup, the same hash function is applied to the search value, the bucket is computed, and that bucket page is read directly. This gives $O(1)$ average lookup time — only 2–3 page reads regardless of table size. PostgreSQL's implementation uses dynamic expansion: when the load factor exceeds a threshold, the number of buckets doubles and entries are redistributed.

Q2. Why can't hash indexes support range queries?

A: Hash functions deliberately destroy ordering — two similar keys (e.g., 100 and 101) hash to completely different buckets with no predictable relationship. A range query like `price > 100` requires knowing all values greater than 100, but hash buckets contain values in no meaningful order. There is no way to find a "starting point" and scan forward. B-tree indexes work for ranges because keys are stored in sorted order within and across leaf pages.

Q3. Before PostgreSQL 10, why were hash indexes dangerous to use?

A: Before PG10, hash index operations were not written to the Write-Ahead Log (WAL). This had two consequences: (1) Crash recovery — if the server crashed, hash indexes were not recoverable and had to be

rebuilt manually after recovery. (2) Replication — since changes were not in WAL, streaming replication did not replicate hash index changes to standby servers, making standbys have outdated or corrupt hash indexes. From PG10 onward, hash indexes are fully WAL-logged and safe for production.

Q4. When would you choose a hash index over a B-tree?

A: Hash indexes are worth considering when: (1) The column has long values (e.g., URLs, SHA hashes, long tokens) because the hash entry (fixed ~10 bytes) is much smaller than a B-tree entry that stores the full key value. (2) All queries on this column are strict equality lookups — no ranges, no ordering, no prefix matching. (3) The smaller index size matters for storage or cache pressure. In practice, the use case is narrow and B-tree is preferred for most columns because it supports all operator types.

Q5. Can you create a UNIQUE hash index?

A: No. PostgreSQL does not support unique hash indexes. UNIQUE constraints always use B-tree. This is because enforcing uniqueness requires checking whether a value already exists, which for a unique constraint must be reliable and efficient, and B-tree provides the necessary structure for that.

Q6. How does a hash index handle overflow pages?

A: When a bucket page fills up and a new entry must be inserted into that bucket, an overflow page is allocated and chained to the bucket page via a pointer. If the overflow page also fills up, another overflow page is chained. Lookups must follow the overflow chain to find all matching entries. PostgreSQL manages a bitmap of free overflow pages to reuse them when entries are deleted. If overflow chains become long, it indicates the load factor is too high and the index should be rebuilt.

Q7. How does hash index size compare to B-tree for VARCHAR(200) columns?

A: For a VARCHAR(200) column with average 150-character values: each B-tree leaf entry stores the full key value (~150 bytes + ctid + overhead = ~160 bytes), while each hash entry stores only the hash value (4 bytes) + ctid (6 bytes) = ~10 bytes. The hash index would be approximately 16× smaller for this key type. For a B-tree index the key must be stored to enable range comparisons, while for a hash index only the hash bucket is needed for equality comparison.

Exercises with Solutions

Exercise 1

A `url_shortener` table stores `short_code` (VARCHAR(8)) and `long_url` (TEXT, up to 2000 chars). Queries look up `long_url` by exact value to check for duplicates. Should you use B-tree or hash for the `long_url` index?

Solution: Hash index is the better choice because:

- Queries are strictly equality (`WHERE long_url = ?`)
- `long_url` is very long (up to 2000 chars), making hash entries ~40× smaller
- No range queries, no ordering needed on `long_url`

```
CREATE INDEX CONCURRENTLY idx_urls_long_url USING hash ON url_shortener(long_url);
```

```
-- Verify it's used:
```

```
EXPLAIN SELECT short_code FROM url_shortener
WHERE long_url = 'https://very-long-url.example.com/...';
```

Exercise 2

A `sessions` table has a `token` column (VARCHAR(64), random UUID-like). Write a query to compare the size of a B-tree vs hash index on this column (after creating both).

Solution:

```
-- Create both indexes
CREATE INDEX idx_sessions_token_btree ON sessions(token);
CREATE INDEX idx_sessions_token_hash USING hash ON sessions(token);

-- Compare sizes
SELECT
    indexname,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size,
    amname AS index_type
FROM pg_stat_user_indexes
JOIN pg_index USING (indexrelid)
JOIN pg_class idx ON idx.oid = indexrelid
JOIN pg_am ON idx.relam = pg_am.oid
WHERE relname = 'sessions'
    AND indexname IN ('idx_sessions_token_btree', 'idx_sessions_token_hash');

-- Drop the larger one after comparison
DROP INDEX idx_sessions_token_btree; -- likely larger for long tokens
```

Production Scenarios

Scenario 1: Authentication Token Lookup

A web application stores JWT tokens in a `user_sessions` table (50M rows). Token lookup on every HTTP request is the most frequent query.

```
-- Column: token TEXT (250+ chars JWT)
-- Query pattern: WHERE token = $1 (exact match only)

-- Hash index is ideal here:
CREATE INDEX CONCURRENTLY idx_user_sessions_token
USING hash ON user_sessions(token);

-- Result:
-- B-tree size: ~12 GB
-- Hash size: ~750 MB (16x smaller for long text)
-- Lookup time: ~0.05ms (3 page reads vs 4-5 for B-tree)
```

Scenario 2: Content Deduplication

A file storage system needs to check if a file (identified by SHA-256 hash, a 64-char hex string) already exists.

```
CREATE TABLE stored_files (  
    id BIGSERIAL PRIMARY KEY,  
    sha256 CHAR(64) NOT NULL,  
    path TEXT,  
    size BIGINT  
);  
  
-- Hash index for deduplication check  
CREATE INDEX CONCURRENTLY idx_files_sha256 USING hash ON stored_files(sha256);  
  
-- Deduplication query:  
SELECT id, path FROM stored_files  
WHERE sha256 = '9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a08';  
-- Uses hash index: 2-3 page reads, sub-millisecond
```

Cross-References

- [01 index fundamentals.md](#) — Index types overview
- [02 btree indexes.md](#) — B-tree for comparison
- [../08 Query Optimization/03 scan types.md](#) — Index Scan mechanics